From Recursion to Loop: `BinarySearchLoop`

Practice: nonrecursive binary search (Exercise 4.2.2), step by step

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026 · Sedgewick & Wayne, *Introduction to Programming in Java* 2e, §4.2

Why this exercise

- **Your first recursion-to-loop rewrite.** Binary search is just the example: you will need this again for mergesort, stacks (4.3), and recursions that get too deep
- **The half-open interval `a[lo, hi)` made concrete.** The loop condition, the new `lo` or `hi`, and the `-1` after the loop are all questions about the interval
- **What real code does.** Java's own `Arrays.binarySearch` is a loop. Loop invariant: if `key` is in `a`, it is in `a[lo, hi)`
- **Binary search itself doesn't need it.** The recursion is only about lg N deep – about 30 for a billion entries
- **Do it by hand, without AI.** Any chatbot writes this in seconds; the point is that you can

Practice: BinarySearchLoop (15 min)

→ **Switch to editor · BinarySearchLoop.java, test client from class**

```
public static int search(String key, String[] a)
{
    return search(key, a, 0, a.length);
}
public static int search(String key, String[] a,
                         int lo, int hi)
{
    if (hi <= lo) return -1;
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) return search(key, a, lo, mid);
    else if (cmp < 0) return search(key, a, mid+1, hi);
    else              return mid;
}
```

Task: write one method `search(String key, String[] a)` that does the same with a `while` loop instead of recursion.

We build it in four steps. Write one line per step, then compare with your neighbor.

Closed book, no AI.

Step 1 · Which parameters change, and where do they start?

```
return search(key, a, 0, a.length);    // wrapper
...
if (cmp > 0)      return search(key, a, lo, mid);
else if (cmp < 0) return search(key, a, mid+1, hi);
```

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
}
```

Compare the calls with the parameters `(key, a, lo, hi)`. Which change? Where do they start?

→ `key` and `a` stay the same; `lo` and `hi` change. They become the loop variables.

→ Their first values come from the wrapper: `0` and `a.length`, the whole array.

Step 2 · Base case → loop condition

```
if (hi <= lo) return -1;
```

The recursion stops here. Turn it around: as long as what is true should the loop keep going?

→ `while (hi > lo)` – the interval is not empty. Strict >: `lo == hi` is the empty interval.

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
    while (hi > lo)
    {
    }
}
```

Step 3 · Calls → assignments

	key	a	lo	hi
current call	key	a	lo	hi
search(key, a, lo, mid)	key	a	lo	**mid**
search(key, a, mid+1, hi)	key	a	**mid+1**	hi

→ Only one position changes per call: `hi = mid;` and `lo = mid + 1;` No temporary variable needed.

→ `return mid;` is not a call: it stays, inside the loop. It ends the loop and the method at once.

```
int lo = 0, hi = a.length;
while (hi > lo)
{
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) hi = mid;
    else if (cmp < 0) lo = mid + 1;
    else              return mid;
}
```

Step 4 · Base return → after the loop

The loop has two ways out. Which one leads to the line after the loop, and what goes there?

→ `return mid` leaves directly (found). If the condition turns false, the interval is empty: not found. That's the recursion's base case, so its return goes after the loop: `return -1;`

Now run the test client from class.

```
int lo = 0, hi = a.length;
while (hi > lo)
{
    int mid = lo + (hi - lo) / 2;
    int cmp = a[mid].compareTo(key);
    if      (cmp > 0) hi = mid;
    else if (cmp < 0) lo = mid + 1;
    else              return mid;
}
return -1;
```

Same tests, same results

```
public static int search(String key, String[] a)
{
    int lo = 0, hi = a.length;
    while (hi > lo)
    {
        int mid = lo + (hi - lo) / 2;
        int cmp = a[mid].compareTo(key);
        if      (cmp > 0) hi = mid;
        else if (cmp < 0) lo = mid + 1;
        else              return mid;
    }
    return -1;
}
```

Test client from class:

```
all 15 hits found
aaron  -> -1
marvin -> -1
zoe    -> -1
empty, eve -> -1
one,   eve -> 0
one,   bob -> -1
```

→ Identical to the recursive version.

Why does the book use recursion?

Same authors, two books, two versions – why?

- **The book teaches.** The recursive code says what the algorithm does: look at the middle, then search the left or the right half
- **Analysis and proof follow the code.** $T(N) = T(N/2) + 1$ (p. 535): one compare plus one call on half the interval. Correctness: induction on the interval size
- **It fits the chapter.** 4.2 builds on recursion (2.3), and mergesort comes next. Exercise 4.2.2 asks for the loop, so the book wants you to see both
- **The library has to be fast.** Every call costs a stack frame, and Java does not turn a call that comes last into a loop by itself. Not because of stack overflow: the depth is only lg N
- Sedgewick & Wayne, *Algorithms* 4e – a book about efficient implementations – writes `BinarySearch` with a `while` loop